ICMC USP SÃO CARLOS



Análise Assintótica

Conceitos Básicos

Sumário

- Conceitos Básicos
- Função de Complexidade
- Ordem de Crescimento

Referências

- Aula baseado no capítulo “Introdução à Análise Assintótica” de SERPA, Matheus S.; RODRIGUES, Thiago N.; ALVES, Ítalo C.; et al. Análise de Algoritmos. Porto Alegre: SAGAH, 2021. E-book. ISBN 9786556901862. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9786556901862/
- CORMEN, Thomas. Algoritmos - Teoria e Prática. 3rd ed. Rio de Janeiro: GEN LTC, 2012. E-book. p.32. ISBN 9788595158092. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9788595158092/

Conceitos Básicos

- Algoritmos devem fornecer soluções para problemas computacionais.
- Requisitos necessários:
  - Corretude: para cada entrada, retorna a saída correta.
  - Eficiência: velocidade de execução e uso do espaço.

Conceitos Básicos

- Os primeiros computadores apresentavam restrições severas quanto aos recursos de tempo e espaço.
- Algoritmos implementados em máquinas atuais não demandam uma quantidade crítica de espaço extra para sua execução.

Conceitos Básicos

- A eficiência relativa ao tempo de execução não evoluiu na mesma proporção.
- Por isso, a análise de algoritmos se concentrou principalmente na eficiência do tempo.
  - Complexidade de tempo no pior caso

Conceitos Básicos

- Relembrando a busca sequencial:
  - Entrada: L (lista), e (elemento da busca)
  - Saída: True (encontrado) ou False.
  - N= len(L) - tamanho da entrada;

```
3  def busca(L,e):
4      for i in range(len(L)):
5          if L[i] == e:
6              return True
7      return False
```

Conceitos Básicos

- Complexidade de tempo no pior caso ocorrendo quando o elemento não está na lista ou é o último elemento.
- A complexidade de tempo do pior caso será linear em N.
  - Proporcional ao tamanho da lista

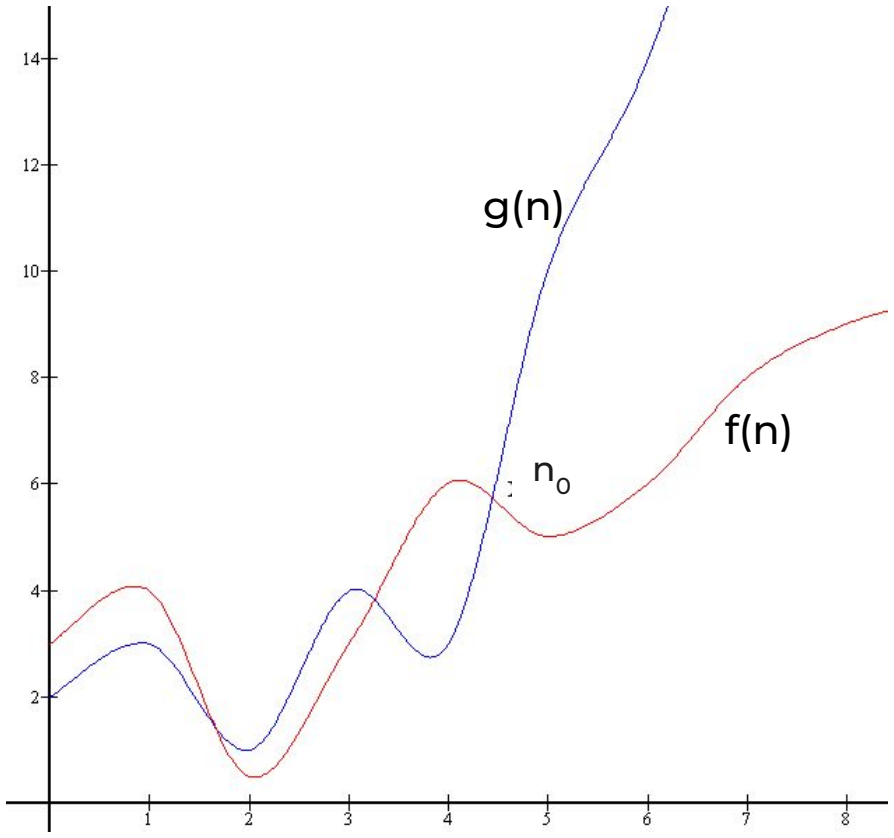
```
3  def busca(L,e):
4      for i in range(len(L)):
5          if L[i] == e:
6              return True
7      return False
```

Conceitos Básicos

- Relembrando, o tempo de execução está vinculado ao tamanho das instâncias do problema computacional.
- O tempo de processamento cresce proporcionalmente ao tamanho do problema.
- Quanto o tempo cresce baseado no aumenta do tamanho da instância?

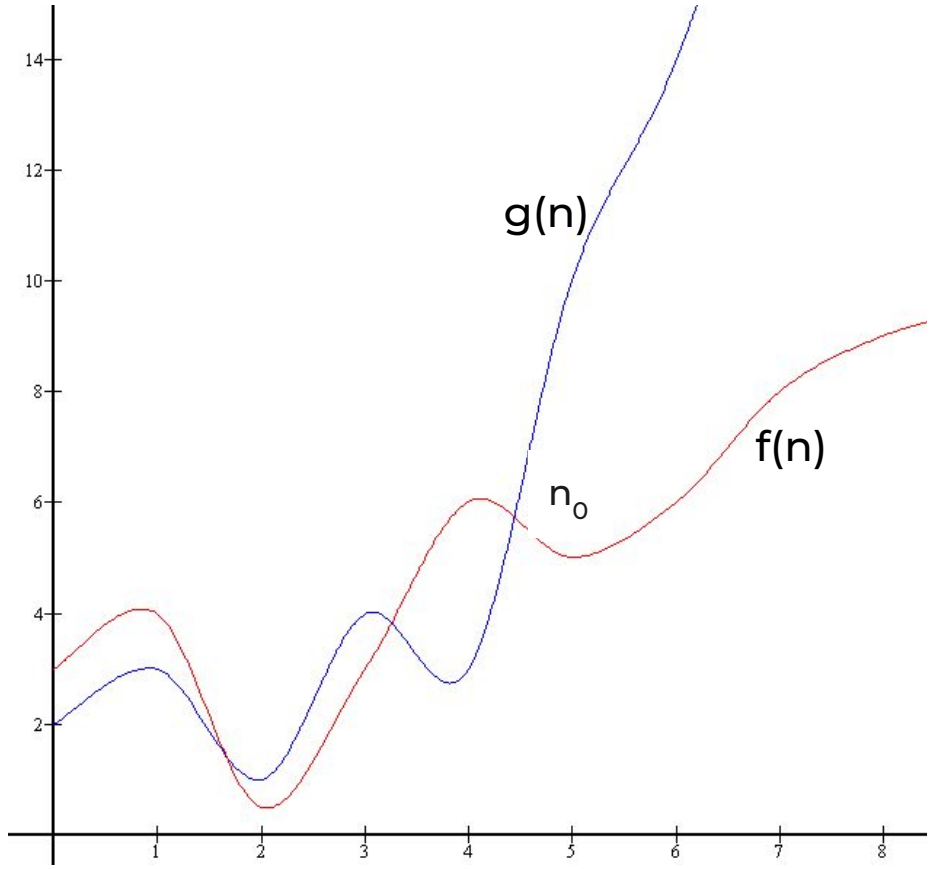
Função de Complexidade

- A análise assintótica faz uma análise de eficiência do algoritmo estabelecendo uma função f(n) que relaciona o tamanho da instância n ao custo em termos de tempo computacional.
  - f(n): função custo ou função de complexidade.



Função de Complexidade

- Para instâncias com tamanho pequeno, o custo retornado por g(n) indica um tempo computacional reduzido ainda que o algoritmo não seja o mais eficiente.



Função de Complexidade

- A análise assintótica se concentra no comportamento do algoritmo para instâncias grandes.
- A função f(n) avalia a complexidade de tempo do algoritmo quando n cresce.

Entrada	N	N log N	N^2	N^3	2^N	N!
10	< 1 seg	< 1 seg	< 1 seg	< 1 seg	< 1 seg	4 seg
30	< 1 seg	< 1 seg	< 1 seg	< 1 seg	18 min	10^{25} anos
50	< 1 seg	< 1 seg	< 1 seg	< 1 seg	36 anos	Muito tempo
100	< 1 seg	< 1 seg	< 1 seg	1 seg	10^{17} anos	Muito tempo
1 000	< 1 seg	< 1 seg	1 seg	18 min	Muito tempo	Muito tempo
10 000	< 1 seg	< 1 seg	2 min	12 dias	Muito tempo	Muito tempo
100 000	< 1 seg	2 seg	3 horas	32 anos	Muito tempo	Muito tempo
1 000 000	1 seg	20 seg	12 dias	31 710 anos	Muito tempo	Muito tempo

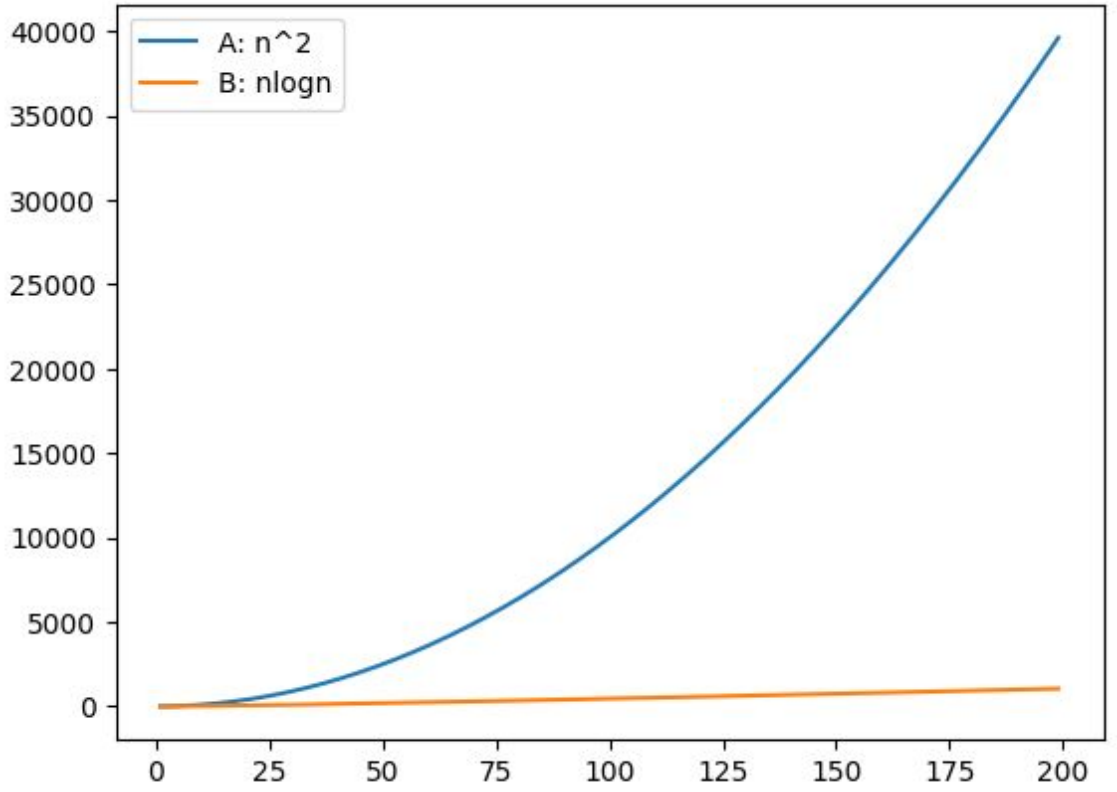
Fonte: Adaptado de Kleinberg e Tardos (2006).

Função de Complexidade

- Logo, busca-se estabelecer funções de custo para problemas computacionais com instâncias grandes.
- O comportamento assintótico de um função de custo f(n) para um algoritmo passa a ser analisado.
- A análise assintótica avalia o comportamento de funções quando suas entradas tendem a valores limites como o infinito.

Função de Complexidade

- Avalia-se o comportamento assintótico das funções de custo dos algoritmos.
- Assim, obtém-se o limite do comportamento do custo dos algoritmos quando n cresce



Função de Complexidade

Complexidade	Descrição
Constante $f(n) = 1$	O desempenho de algoritmos dessa classe não é afetado pelo tamanho da entrada. Nesse caso, as instruções do algoritmo são executadas um número fixo de vezes.
Logarítmica $f(n) = \log(n)$	Esse tempo de execução descreve algoritmos cujo funcionamento é baseado na conversão do problema original em problemas menores.
Linear $f(n) = n$	Algoritmos dessa classe, em geral, realizam alguma operação sobre cada elemento da entrada. Para cada acréscimo no tamanho da entrada, o tempo de execução aumenta correspondentemente.
Linearítmica $f(n) = n \log(n)$	A eficiência de algoritmos que implementam a estratégia de dividir e conquistar costuma ser descrita por essa função de complexidade. Nessa estratégia, o problema é fracionado em instâncias menores (dividir), cada instância é resolvida de forma independente e, na sequência, as soluções são combinadas para gerar a solução do problema original (conquistar).

Fonte: Adaptado de Ziviani (1999).

Função de Complexidade

Complexidade	Descrição
Quadrática $f(n) = n^2$	Algoritmos com essa complexidade processam os dados da entrada, em geral, aos pares. É comum apresentarem um aninhamento de laços de execução, ou seja, um laço sendo executado dentro de outro laço.
Cúbica $f(n) = n^3$	Algoritmos desse tipo são geralmente utilizados apenas em problemas de pequeno porte. Isso porque programas que apresentam essa ordem de crescimento, em geral, são compostos de três laços aninhados, usados para alguma operação envolvendo todas as triplas de n elementos.
Exponencial $f(n) = c^n,\ c > 1$	Algoritmos com essa função de complexidade não têm aplicabilidade em problemas práticos. Isso porque eles implementam uma estratégia de força bruta (avaliar todas as soluções possíveis) para resolver um problema.

Fonte: Adaptado de Ziviani (1999).

Função de Complexidade

- Comportamento assintótico das funções de complexidade das principais classes de problemas computacionais.

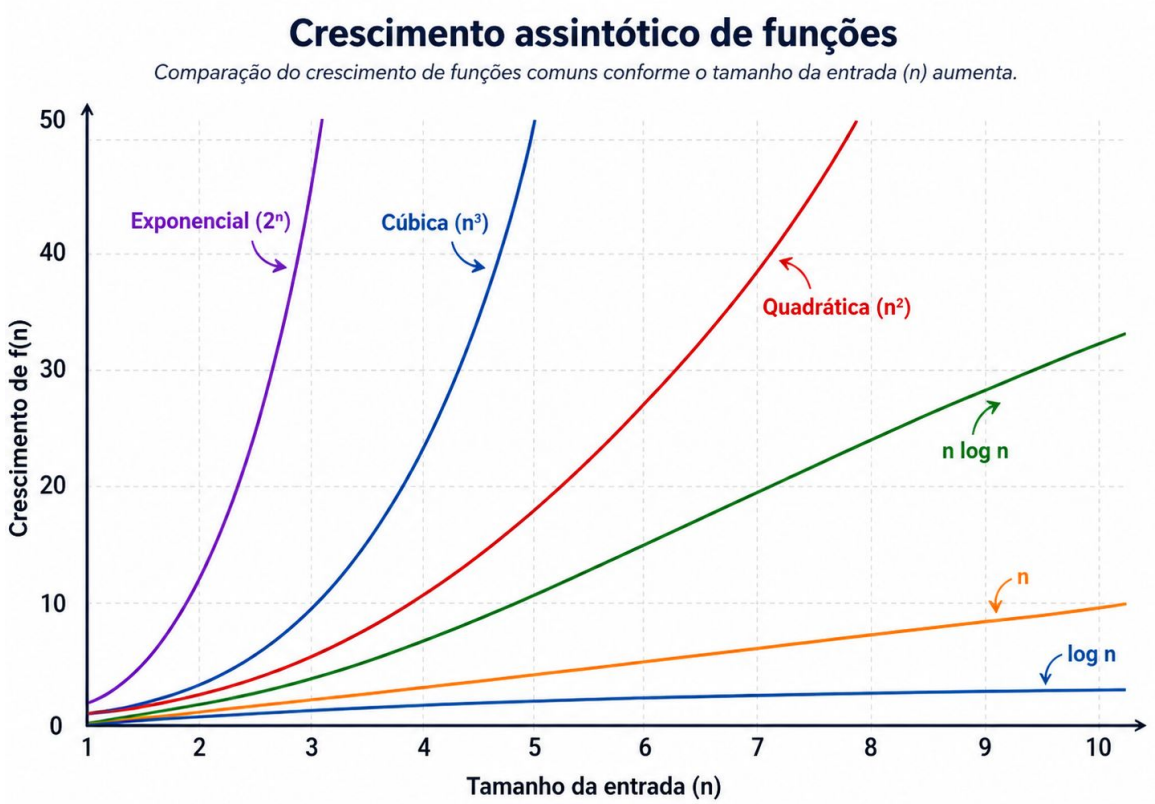
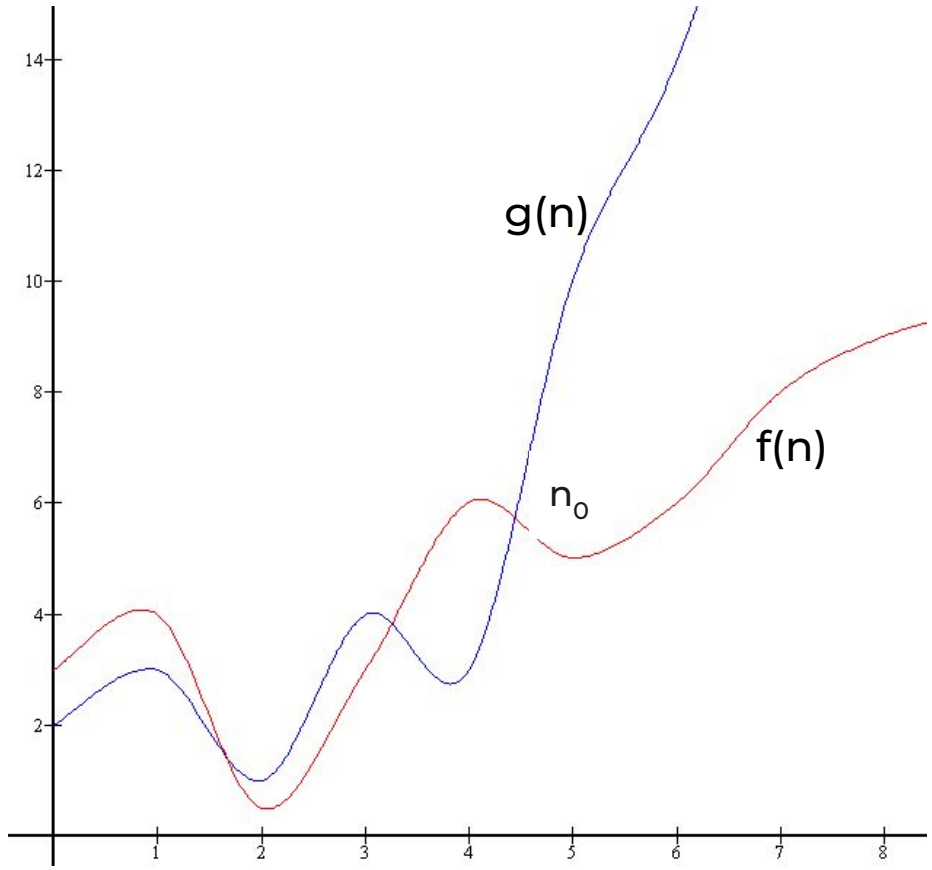


Figura 1. Hierarquia assintótica de funções.

Ordem de Crescimento

- Na maioria dos casos, deseja-se obter um limitante superior para o crescimento do tempo de execução em função do tamanho da entrada no pior caso.
- Para isso, vamos comparar duas funções de custo f(n) e g(n), onde g(n) desempenha o papel de limitante superior para o crescimento do tempo de execução dado por f(n).



Ordem de Crescimento

- Vamos assumir f(n) e g(n) como funções definidas no domínio dos números naturais.
- O limitante superior será assintoticamente definido solucionando o seguinte problema:
  - Determinar uma constante tal que $f(n) \leq c\,g(n)$ para todo n suficientemente grande.

